

1.4 Data Types, Data Structures and Boolean Algebra

Computer Science — OCR — Component 01 (Computer Systems) — Spec sections 1.4.1, 1.4.2, 1.4.3

This is the largest single topic in H446 Paper 1. It combines three distinct skills: **number representation** (binary, hex, two's complement, floating-point), **abstract data structures** (arrays, lists, stacks, queues, trees, graphs, hash tables) and **Boolean algebra** (logic gates, identities, K-maps, adders, flip-flops). Every exam series contains several questions from this topic, ranging from short 1-mark conversions to full 8-mark simplification or BST-insertion questions.

1.4.1 Data Types

Primitive Data Types

A **data type** specifies the kind of value a variable may hold, the operations that may be performed on it, and the way it is stored in memory. A **primitive data type** is one provided directly by the language (not composed of other types).

OCR H446 recognises five primitive data types. You must know their typical storage size, the set of values they can represent, and one or two realistic use cases.

Type	Typical size	Range / values	Example	Use
Integer	2, 4 or 8 bytes (16/32/64 bits)	Whole numbers. 32-bit signed: −2,147,483,648 to +2,147,483,647. 32-bit unsigned: 0 to 4,294,967,295.	<code>age = 17</code>	Counters, indices, exact integer maths.
Real / Float	4 bytes (single) or 8 bytes (double)	Approximate fractional numbers using IEEE 754. Single: $\sim \pm 3.4 \times 10^{38}$; double: $\sim \pm 1.8 \times 10^{308}$.	<code>pi = 3.14159</code>	Scientific calculations, graphics, measurements.
Character	1 byte (ASCII) or 2–4 bytes (Unicode)	A single symbol: letter, digit, punctuation or control code.	<code>grade = 'A'</code>	Individual keystrokes, menu choices.
String	Variable (1 byte per char + length/null)	Ordered sequence of characters.	<code>name = "Alice"</code>	Names, messages, filenames — any textual data.

Boolean	1 bit conceptually; typically 1 byte in memory	Exactly two values: <code>true</code> or <code>false</code> .	<code>isAdult</code> <code>= true</code>	Flags, loop conditions, logical decisions.
----------------	--	--	---	--

Exam Tip: Although a Boolean only *needs* one bit, most languages store it as a whole byte because the smallest addressable unit of memory is a byte. If asked "how much memory does a Boolean really require?" the answer is "1 bit of information but typically 1 byte of storage".

Why Data Types Matter

- **Memory allocation** — the compiler reserves the correct amount of memory.
- **Permitted operations** — you can add two integers but not two booleans.
- **Interpretation of bits** — the bit pattern `01000001` represents 65 as an integer, 'A' as an ASCII character, or a completely different, tiny value (about 9.1×10^{-44}) if padded to 32 bits and read as an IEEE 754 float.
- **Error detection** — type mismatches caught at compile time prevent run-time bugs.

The Binary Number System

Binary is a base-2 positional number system using only the digits 0 and 1. Each digit position represents a power of 2. Binary is used internally by all digital computers because transistors have two stable states (on/off, high/low voltage) which can directly encode 0 and 1.

Place-Value Table — Powers of 2

Bit position	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Denary	32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1

Memorise at least the first 10 powers of 2 (up to 1024). The columns double each time you move left.

$2^{10} = 1024$ is the key "K" boundary used in KiB/MiB/GiB.

Denary → Binary Conversion (Two Methods)

Method 1: Repeated subtraction from a place-value table. Write out the powers of 2; work left to right subtracting each when possible.

Convert **173** to 8-bit binary:

```
Power: 128 64 32 16 8 4 2 1
173 - 128 = 45 → 1 in the 128 column
45 - 64 = - → 0 (64 won't go)
45 - 32 = 13 → 1 in the 32 column
```

```

13 - 16 = - → 0
13 - 8 = 5 → 1
5 - 4 = 1 → 1
1 - 2 = - → 0
1 - 1 = 0 → 1

```

Result: 10101101

Check: $128+32+8+4+1 = 173$ ✓

Method 2: Repeated division by 2. Divide by 2 repeatedly, reading the remainders *bottom-to-top*.

```

173 ÷ 2 = 86 r 1 ← LSB
86 ÷ 2 = 43 r 0
43 ÷ 2 = 21 r 1
21 ÷ 2 = 10 r 1
10 ÷ 2 = 5 r 0
5 ÷ 2 = 2 r 1
2 ÷ 2 = 1 r 0
1 ÷ 2 = 0 r 1 ← MSB

```

Read upwards: 10101101

Binary → Denary Conversion

Write each bit under its place value and add the columns where a 1 appears.

```

  1  1  0  0  1  0  1  1
128 64 32 16  8  4  2  1
128+64+ 0+ 0+ 8+ 0+ 2+ 1 = 203

```

Exam Tip: A common 1-mark trap: always check the **number of bits requested**. "Express 13 as an 8-bit binary number" requires `00001101`, not `1101`. Leading zeros carry marks.

Unsigned vs Signed

An **unsigned** n -bit number represents 0 to 2^n-1 . An **8-bit** unsigned integer holds 0 to 255. Signed representations sacrifice one value-bit for a sign, giving -128 to +127 in two's complement.

Hexadecimal (Base 16)

Hexadecimal is a base-16 positional number system using the sixteen digits 0–9 and A–F (where A=10, B=11, C=12, D=13, E=14, F=15). Each hex digit represents exactly **four binary digits (one nibble)**, making hex a compact human-readable shorthand for binary.

Why Use Hex?

- **Compactness** — a 32-bit pattern takes 8 hex digits instead of 32 binary digits.
- **Easy conversion** — no maths: group bits into nibbles and translate.
- **Error-resistant** — fewer digits to read or type reduces mis-transcription in memory addresses, machine code, colour codes and MAC addresses.
- **Conventional** — used for HTML colours (#FF5733), IPv6 addresses, assembly language, debugger output.

Hex Digit Table

Denary	Binary	Hex	Denary	Binary	Hex
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

Binary ↔ Hex (via nibbles)

Convert 10110110_2 to hex:

```
Split into nibbles from the right:  1011 | 0110
Translate each nibble:               B   |   6
Result: B6
```

Convert $2F_{16}$ to binary:

```
2 → 0010
F → 1111
Concatenate: 00101111
```

Denary ↔ Hex

Convert 218_{10} to hex via binary:

```
218 = 128+64+16+8+2 → 11011010
Split:  1101 1010
```

```
Hex:      D    A
Result: DA
```

Convert $4F_{16}$ to denary directly (place values 16 and 1):

```
4 × 16 = 64
F ×  1 = 15
Sum    = 79
```

Exam Tip: Hex conversion questions are almost always worth 1 or 2 marks — the fastest route is via binary. Never try to divide by 16 under pressure.

Octal (Brief Mention)

Octal is base-8, using digits 0–7. Each octal digit maps to exactly three binary digits. Octal is rarely used today; its main legacy uses are Unix file permissions (`chmod 755`) and some older microcontroller assemblers. It is mentioned in the OCR spec for completeness but is not an exam focus.

```
Binary 110 101 111 → Octal 6 5 7 → 6578
```

Signed Binary Representations

Sign-and-Magnitude

Sign-and-magnitude uses the **most significant bit (MSB) as a sign bit** (0 = positive, 1 = negative) and the remaining bits to encode the magnitude in ordinary unsigned binary.

With n bits, sign-and-magnitude represents values in the range $-(2^{n-1}-1)$ to $+(2^{n-1}-1)$. For 8 bits this is -127 to $+127$.

Examples (8 bits):

```
+13 → 0 0001101
-13 → 1 0001101
+0  → 0 0000000
-0  → 1 0000000  (!)
```

Problems with sign-and-magnitude:

- **Two representations of zero** (+0 and -0) — wastes a bit pattern and complicates equality tests.
- **Addition is awkward** — the CPU must inspect the sign bits, compare magnitudes, and decide whether to add or subtract. This requires extra hardware.

Because of these problems, modern CPUs do **not** use sign-and-magnitude for integer arithmetic.

Two's Complement

Two's complement is the dominant representation for signed integers. The MSB has **negative place value**: for 8 bits the column weights are $-128, 64, 32, 16, 8, 4, 2, 1$. This gives a single zero and allows normal binary addition hardware to handle signed arithmetic with no modification.

For n bits, two's complement represents -2^{n-1} to $+2^{n-1}-1$. For 8 bits: **-128 to +127**.

Converting a Negative Denary to Two's Complement

Two methods — both accepted in exams.

Method A: Flip and add 1. Convert $+X$ to binary, flip all bits, add 1.

Convert -45 to 8-bit two's complement:

+45 in binary: 00101101
 Flip (one's complement): 11010010
 Add 1: 11010011

Check (column sums): $-128 + 64 + 16 + 2 + 1 = -45$ ✓

Method B: Negative place value. Use the -128 column directly.

$-45 = -128 + 83$
 $83 = 64 + 16 + 2 + 1 \rightarrow 1010011$
 So $-45 = 1\ 1010011$ (the -128 column is set)
 $= 11010011$

Reading a Two's Complement Number

Interpret **10110101** as a signed 8-bit value:

Columns: $-128\ 64\ 32\ 16\ 8\ 4\ 2\ 1$
 Bits: 1 0 1 1 0 1 0 1
 Sum: $-128 + 0 + 32 + 16 + 0 + 4 + 0 + 1 = -75$

Exam Tip: The fastest exam check: if the MSB is 1, the number is negative. Flip all bits, add 1, convert to denary, put a minus sign. For **10110101** : flip $\rightarrow$ **01001010** , $+1 \rightarrow$ **01001011** = 75, so the original is -75 .

Range and Advantages of Two's Complement

Bits	Unsigned range	Two's complement range
4	0 to 15	-8 to +7
8	0 to 255	-128 to +127

16	0 to 65,535	−32,768 to +32,767
32	0 to 4,294,967,295	−2,147,483,648 to +2,147,483,647

Advantages over sign-and-magnitude:

- **Single zero** — only `00000000`, freeing one extra negative value.
- **Simpler hardware** — ordinary binary adder works for signed addition and subtraction.
- **Subtraction = addition of complement** — the CPU reuses the adder.

Binary Addition

Rules

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1
1	1 + carry 1	1	1

Worked Example

```

Add   01011010  (90)
      + 00110111  (55)
      -----
Carry:1111 11
      01011010
      + 00110111
      -----
      10010001  (145)  ✓

```

Overflow

Overflow occurs when the result of an arithmetic operation cannot be represented within the fixed number of bits available. In two's complement, overflow is detected when **the carry into the sign bit differs from the carry out of the sign bit**, equivalently when two positives sum to a negative or two negatives sum to a positive.

Adding two positive 8-bit two's complement numbers: 100 + 50.

```

    01100100  (+100)
+   00110010  (+50)
-----
    10010110  → interpreted as -106 (!)

```

Two positives produced a negative result. The true answer 150 lies outside the $-128..+127$ range — **overflow detected**. The V flag in the status register would be set.

Two's Complement Overflow Detection

Carry

into MSB $\neq$ carry out of MSB $\Rightarrow$ Overflow A B Sum Carry 0 1 1 0 0 1 0 0 0 1 0 1 0 0 1 0 1 0 0 1 0 1 1 0 C_out=0 C_in=1 sign bit = +100 = +50 = -106 (wrong!) C_out $\neq$ C_in $\Rightarrow$ OVERFLOW Two positives produced a negative: the V (overflow) flag is set in the Status Register.

Exam Tip: Distinguish **carry** from **overflow**. Carry is set when *unsigned* arithmetic produces a result $> 2^n - 1$ (a carry out of the MSB). Overflow is set when *signed* arithmetic leaves the two's complement range. In sign-less contexts you use C; in signed contexts you use V.

Binary Subtraction via Two's Complement

To subtract B from A, negate B (flip + add 1) and add to A. The carry out of the MSB is ignored — the result sits in the n result bits.

Compute 57 – 22 in 8-bit two's complement:

```

57          = 00111001
22          = 00010110
-22 (flip+1) = 11101010

  00111001
+ 11101010
-----
1 00100011  (ignore the 1 that falls off the top)
  00100011  = 35  ✓

```

Binary Multiplication — Shift-and-Add

Multiplying by 2 shifts left by 1 (appending a 0). Multiplying by 2^k shifts left by k . General multiplication uses the shift-and-add algorithm: for each 1 in the multiplier, add a left-shifted copy of the multiplicand.

```

13 × 5 in binary: 1101 × 0101

  1101      ← 1101 × 1 (bit 0 of 0101)
+ 0000      ← 1101 × 0, shifted left 1
+ 110100    ← 1101 × 1, shifted left 2
+ 0000000   ← 1101 × 0, shifted left 3
-----
1000001     = 65 ✓
  
```

Fixed-Point vs Floating-Point

Fixed-Point

Fixed-point representation places the binary point at a **fixed, agreed position** within the bit pattern. For example, an 8-bit fixed-point format with 4 integer bits and 4 fraction bits interprets **0101.1010** as 5.625 ($4 + 1 + 0.5 + 0.125$).

Fraction place values to the right of the point are $2^{-1}=0.5$, $2^{-2}=0.25$, $2^{-3}=0.125$, $2^{-4}=0.0625$, ...

Advantages: **simple, fast**, precise within its range. Disadvantages: **very limited range** — an 8-bit signed fixed-point can only represent values from about -8 to $+8$ in 0.0625 steps.

Floating-Point

Floating-point representation stores a number in scientific form as **mantissa × base^{exponent}**. In binary: value = mantissa × 2^{exponent} . The bit pattern is split into three fields: **sign**, **exponent** and **mantissa** (also called the significand). This trades some precision for a vastly larger range.

Floating-Point Bit Layout (OCR 10-bit example)



$2 = 0.6875$ Exponent = $0011_2 = +3$ Value = $0.6875 \times 2^3 = 5.5$

Normalisation

A floating-point number has many possible representations. **Normalisation** standardises them to maximise precision and give a unique form.

A normalised mantissa in OCR-style two's complement form has the **sign bit different from the first bit after the binary point**. So positive normalised numbers start **0.1...** and negative normalised numbers start **1.0...**. This pushes the mantissa as far left as possible without changing its sign, maximising precision.

Normalise 0.0011010×2^5 : shift the mantissa left until the first bit after the binary point is 1, subtracting 1 from the exponent for each shift.

```
0.0011010 × 2^5
→ 0.0110100 × 2^4
→ 0.1101000 × 2^3 ← normalised (starts 0.1)
```

Normalise a negative: 1.1101000×2^3 — shift left until the first bit after the point is 0.

```
1.1101000 × 2^3
→ 1.1010000 × 2^2
→ 1.0100000 × 2^1 ← normalised (starts 1.0)
```

Worked Example: Decimal to Normalised Floating-Point (6-bit mantissa, 4-bit exponent)

Represent +5.5 in the OCR 10-bit floating-point format.

```
Step 1 Convert 5.5 to fixed-point binary:
      5.5 = 4 + 1 + 0.5 = 101.1

Step 2 Express with an explicit binary point in the mantissa:
      101.1 = 0.1011 × 2^3

Step 3 Check normalisation: sign bit 0, first fraction bit 1. ✓
      Mantissa needs 6 bits: 0.10110 0 (pad with trailing zeros)
      Mantissa bit pattern: 010110 0 → 010110
      Exponent = 3, in 4-bit two's complement: 0011

Step 4 Combine:   Mantissa 010110   Exponent 0011
```

Represent -6.75.

```
6.75 = 110.11 in binary
-6.75 in 2's-complement with the point after bit 1 must start 1.0...

Work with the positive first:   +6.75 = 0.11011 × 2^3
Two's complement the mantissa to negate:
    0.11011 flip → 1.00100 + 0.00001 = 1.00101
Check: 1.00101 represents -1 + 0.00101 = -1 + 0.15625 = -0.84375
Value = -0.84375 × 2^3 = -6.75 ✓
```

Pad mantissa to 6 bits: 100101
 Exponent = 3 = 0011

Range, Precision and Errors

- **Range** is set by the **exponent**. More exponent bits $\Rightarrow$ larger range of magnitudes.
- **Precision** is set by the **mantissa**. More mantissa bits $\Rightarrow$ more significant figures.
- **Trade-off**: a fixed total of bits must be split between exponent and mantissa.

Error	Cause	Example
Rounding error	A value has more significant bits than the mantissa can hold; trailing bits are discarded.	0.1 in binary is recurring (0.0001100110011...), stored only to mantissa precision.
Truncation error	Extra bits chopped off rather than rounded.	Similar to rounding but biased downward for positive numbers.
Overflow	Exponent too large to store — result exceeds maximum representable magnitude.	Huge multiplication $\rightarrow$ exponent field cannot hold value.
Underflow	Exponent too small (negative) — number closer to zero than the smallest representable magnitude.	Tiny subtraction of two near-equal numbers.

Exam Tip: Normalisation rule you must memorise: **positive normalised mantissas begin 01, negative normalised mantissas begin 10**. This is the single most commonly tested detail of OCR floating-point questions.

Fixed-Point vs Floating-Point: Trade-offs

Property	Fixed-point	Floating-point
Range	Small, set by bit width	Huge, set by exponent
Precision	Uniform across the range	Relative — fewer sig-figs for larger values
Hardware cost	Low (standard integer ALU)	High (FPU required)
Speed	Faster per operation	Slower but vastly more capable
Typical use	DSPs, embedded control, currency	Scientific, graphics, general-purpose real numbers

Bitwise Operations

Bitwise operators work on individual bits of their operands in parallel.

Operator	Symbol	Rule	Use
AND	&	1 iff both bits 1	Masking — clear specific bits (AND with 0), test bits (AND with mask, compare to 0).
OR		1 iff either bit 1	Set specific bits (OR with 1).
XOR	^	1 iff bits differ	Toggle bits ; simple cryptography; parity; a XOR a = 0 (self-cancelling).
NOT	~	Flip every bit	Generating complements, producing inverse masks.
Left shift	«	All bits move left, zeros fill on right	Multiply by 2 per shift (for unsigned / small signed).
Logical right shift	» or LSR	All bits move right, zeros fill on left	Unsigned divide by 2.
Arithmetic right shift	» or ASR	All bits move right, MSB copied on left (sign extension)	Signed divide by 2 — preserves sign.

Worked Examples

Set bit 3 of 10010000:

```

10010000
OR 00001000
-----
10011000

```

Clear bit 7 of 11001010:

```

11001010
AND 01111111
-----
01001010

```

Toggle bits 0–3 of 10110110:

```

10110110
XOR 00001111
-----
10111001

```

Left shift 00101100 by 2 positions:

```

00101100 → 10110000  (× 4, last two bits may overflow)

```

Logical right shift 11001000 by 1:

```

11001000 → 01100100  (MSB now 0)

```

Arithmetic right shift 11001000 by 1:
 11001000 → 11100100 (sign bit preserved)

Exam Tip: If an OCR question says "right shift" without specifying, assume **logical** for unsigned contexts and **arithmetic** for signed. State which you are using. Arithmetic right shift of 11001000 gives 11100100 ($-56 \div 2 = -28$).

Character Sets

ASCII

ASCII (American Standard Code for Information Interchange) is a **7-bit** character encoding with **128 code points** (0–127). It covers the uppercase and lowercase Latin alphabet, decimal digits, standard punctuation, and **33 non-printing control codes** (e.g. CR, LF, TAB, BEL, NUL, ESC).

The 8th bit was originally left zero or used as a parity bit. Letters are arranged so that simple bit manipulations convert between cases: 'A' = 65 = 1000001, 'a' = 97 = 1100001 — differ only in bit 5.

Extended ASCII

Extended ASCII uses the 8th bit, giving 256 code points. The lower 128 match ASCII; the upper 128 depend on the **code page** (e.g. Windows-1252, ISO-8859-1). This adds accented letters and box-drawing characters but is not internationally standardised — the same byte can mean different characters in different regions.

Unicode

Unicode is a universal character set defining a code point for every character in every human script — currently over 150,000 assigned code points. The **encoding** (UTF-8, UTF-16, UTF-32) determines how the code point is serialised into bytes.

Encoding	Unit size	Code point range	Notes
UTF-8	Variable: 1–4 bytes	U+0000 to U+10FFFF	ASCII-compatible (U+0000–U+007F encode as single ASCII byte). Dominant on the web.
UTF-16	Variable: 2 or 4 bytes (surrogate pairs)	U+0000 to U+10FFFF	Default internal format of Windows and Java strings.
UTF-32	Fixed: 4 bytes	U+0000 to U+10FFFF	Fastest indexing but wasteful for Latin text.

A **BOM (byte-order mark)** at the start of a Unicode file indicates endianness: `FE FF` for UTF-16 BE, `FF FE` for UTF-16 LE, `EF BB BF` for UTF-8.

Looking Up a Character Code

Character 'Z' → ASCII 90 → `0101 1010` → hex 5A
 Character 'a' → ASCII 97 → `0110 0001` → hex 61
 Character '9' → ASCII 57 → `0011 1001` → hex 39

Exam Tip: A classic exam question: why must Unicode be preferred over ASCII for global applications? Answer in terms of **range** (covers every script) and **standardisation** (one meaning for each byte pattern, no code-page ambiguity). Cost: larger file sizes for non-Latin text.

1.4.2 Data Structures

A **data structure** is an organised collection of data values together with the operations defined on them. Choosing the correct structure is often the single biggest factor in the efficiency of an algorithm.

Arrays

An **array** is a **static, contiguous, zero-indexed** block of memory holding a **fixed number** of elements of the **same type (homogeneous)**. Access by index is $O(1)$ because the address of element i is $\text{base} + i \times \text{size}$.

1D Arrays

```
scores = [85, 92, 78, 66, 100]    // 5 elements, indices 0..4
scores[0] → 85
scores[4] → 100
scores[5] → index out of range
```

2D Arrays

A 2D array is a grid indexed by (row, column) and can be thought of as an array of arrays.

```
grid[3][4] → 3 rows, 4 columns

      col 0  col 1  col 2  col 3
row0 [  1,   2,   3,   4 ]
row1 [  5,   6,   7,   8 ]
row2 [  9,  10,  11,  12 ]

grid[1][2] → 7
grid[2][0] → 9
```

Traversal:

```

for r = 0 to 2
  for c = 0 to 3
    print grid[r][c]
  next c
next r

```

Records

A **record** is a **heterogeneous** collection of named **fields** grouped into a single composite data item. Each field has its own name and type.

```

record Student
  studentID : integer
  name      : string
  gpa       : real
  enrolled  : boolean
end record

Student s
s.studentID ← 20471
s.name      ← "Alice Wong"
s.gpa       ← 3.8
s.enrolled  ← true

```

Records are the basis of file records and rows in relational databases.

Lists

A **list** is a dynamic, ordered collection. Unlike arrays, many language implementations allow lists to grow or shrink at runtime. In some languages (Python, JavaScript) lists are heterogeneous.

Operation	Pseudocode	Typical complexity
Append	<code>list.append(x)</code>	Amortised $O(1)$
Insert at index	<code>list.insert(i, x)</code>	$O(n)$ — elements after i shift right
Remove at index	<code>list.remove(i)</code>	$O(n)$
Length	<code>len(list)</code>	$O(1)$
Slice	<code>list[a:b]</code>	$O(b-a)$
Contains	<code>x in list</code>	$O(n)$

Tuples

A **tuple** is an **immutable, heterogeneous, ordered** fixed-length sequence. Once created it cannot be modified.

```
point    ← (3, 7)
person   ← ("Alice", 17, true)
rgb      ← (255, 87, 51)
```

Tuples are ideal for returning multiple values from a function, composite dictionary keys, and constants that must not change.

Static vs Dynamic Data Structures

	Static	Dynamic
Size decided	At compile time	At runtime, can grow/shrink
Memory layout	Contiguous block, known size	Often non-contiguous (nodes linked by pointers)
Example	Array	Linked list, stack, queue, tree
Access time	$O(1)$ random access by index	Often $O(n)$ — traverse pointers
Memory overhead	Low — just the data	Higher — each node stores pointer(s)
Risk	Over-allocation (wasted space) or overflow (too small)	Running out of heap memory; fragmentation
Main advantage	Predictable memory, fast random access, cache-friendly	Flexible size, efficient insertion/deletion at known positions

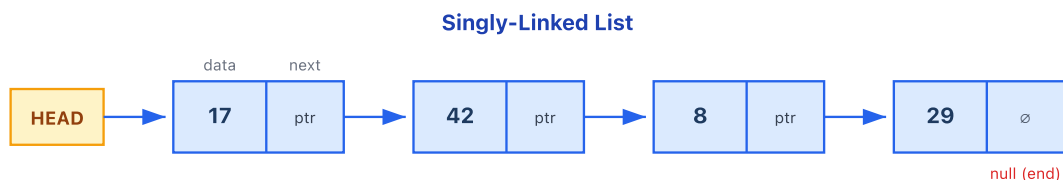
Exam Tip: "State one advantage of a static data structure over a dynamic one" → predictable memory usage or $O(1)$ random access. Don't say "it's faster" without qualifying *what* is faster.

Linked Lists

A **linked list** is a dynamic linear data structure made of **nodes**. Each node holds a **data value** and a **pointer** to the next node. The last node's pointer is `null`. A **head pointer** locates the first node.

Variants:

- **Singly-linked** — each node has one pointer (to next). Traverse forwards only.
- **Doubly-linked** — each node has two pointers (prev and next). Traverse both ways; easier deletion.
- **Circular** — the last node points back to the head (not null). Useful for round-robin scheduling.



Operations

Op	Steps	Complexity
Insert at head	$\text{new.next} \leftarrow \text{head}; \text{head} \leftarrow \text{new}$	$O(1)$
Insert after node p	$\text{new.next} \leftarrow \text{p.next}; \text{p.next} \leftarrow \text{new}$	$O(1)$ given p
Delete node after p	$\text{p.next} \leftarrow \text{p.next.next}$	$O(1)$ given p
Traverse	walk pointers from head until null	$O(n)$
Search	traverse comparing data	$O(n)$

Exam Tip: When inserting into a linked list, always update the new node's pointer *before* redirecting the existing pointer — otherwise the tail of the list is lost.

Queues

A **queue** is a **FIFO (First In, First Out)** abstract data type. Items are added (**enqueued**) at the **rear/tail** and removed (**dequeued**) from the **front/head**.

Real-world uses: print spoolers, CPU process-scheduling round-robin, keyboard input buffer, network packet queues, first-come-first-served checkout simulation.

Operations

Op	Meaning
<code>enqueue(x)</code>	Add x to the rear
<code>dequeue()</code>	Remove and return the front element
<code>front()</code> / <code>peek()</code>	Return the front element without removing
<code>size()</code>	Number of items
<code>isEmpty()</code>	True if queue has 0 items
<code>isFull()</code>	True if a fixed-size queue is full

Linear Queue

FIFO — front pointer advances on dequeue, rear on enqueue

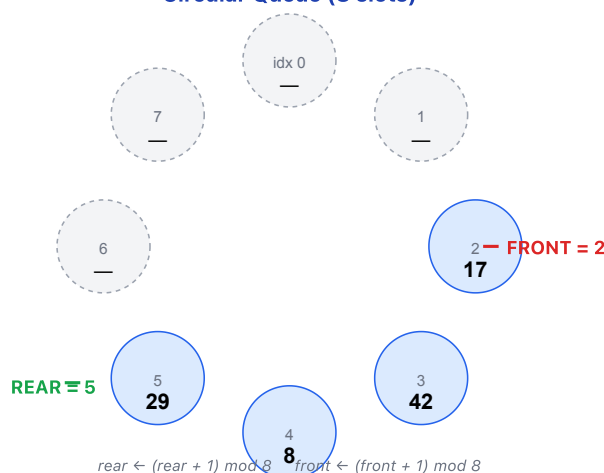


Problem with a simple linear queue: after several dequeues the front pointer has advanced, leaving unused slots at the start that cannot be refilled — the queue appears full before all slots are used. This is solved by a **circular queue**.

Circular Queue

A **circular queue** uses modular arithmetic so that when the rear pointer passes the end of the array it wraps round to the start. Indices are taken modulo the array length: $\text{rear} = (\text{rear} + 1) \bmod \text{size}$.

Circular Queue (8 slots)



Priority Queue

A **priority queue** removes items by **priority rather than arrival order**. Each element carries a priority value; `dequeue()` returns the highest-priority item. Ties are typically broken by arrival order.

Uses: OS process scheduler (real-time tasks first), emergency-room triage simulations, Dijkstra's shortest-path algorithm (which you meet in 2.3).

Stacks

A **stack** is a **LIFO (Last In, First Out)** abstract data type. Items are added (**pushed**) and removed (**popped**) at the same end — the **top**. Only the top element is accessible.

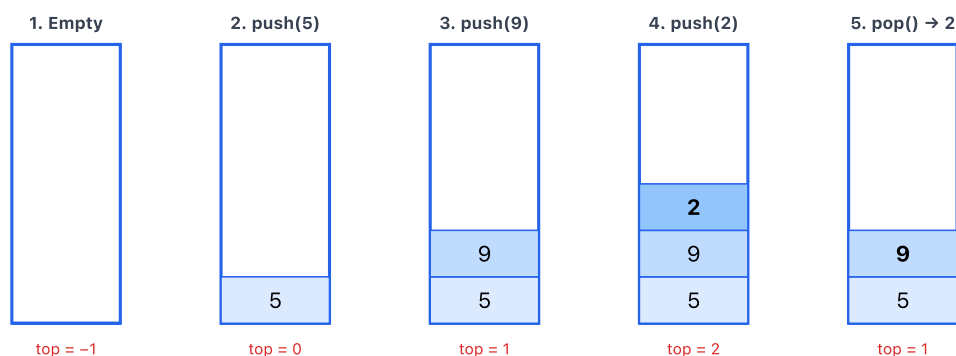
Operations

Op	Meaning
<code>push(x)</code>	Place x on top
<code>pop()</code>	Remove and return top element
<code>peek()</code> / <code>top()</code>	Return top without removing
<code>size()</code>	Number of items
<code>isEmpty()</code>	True if stack empty

Uses

- **Function call stack** — each call pushes a stack frame holding return address, parameters, locals; return pops it.
- **Undo** in editors — pushed actions popped to undo.
- **Backtracking** — maze solvers, parsers.
- **Expression evaluation** and conversion between infix and postfix (Reverse Polish Notation, RPN).
- **Depth-first search** traversal.

Stack Push / Pop Sequence



Reverse Polish Notation

RPN is a **postfix** notation requiring no parentheses, evaluated by a stack. Infix `(3 + 4) * 5` becomes RPN `3 4 + 5 *`.

Evaluate `3 4 + 5 *`

```

push 3    stack: [3]
push 4    stack: [3, 4]
+         pop 4, pop 3, push 3+4=7    stack: [7]
push 5    stack: [7, 5]
*         pop 5, pop 7, push 7*5=35   stack: [35]

```

Result: 35

Trees

A **tree** is a hierarchical, connected, acyclic graph with one designated **root**. Each non-root node has exactly one **parent**; nodes beneath a given node are its **children**. Nodes sharing a parent are **siblings**. A node with no children is a **leaf**. The **depth** of a node is its distance from the root; the **height** of the tree is the depth of the deepest leaf. A **subtree** rooted at any node consists of that node plus all its descendants.

Binary Tree

A **binary tree** is a tree where each node has **at most two children**, conventionally named **left** and **right**.

Binary Search Tree (BST)

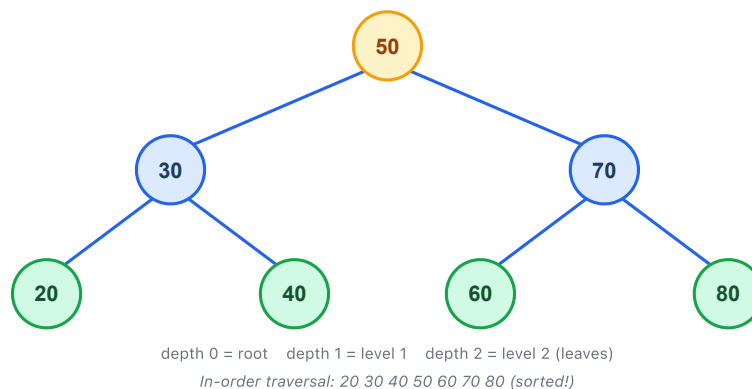
A **Binary Search Tree (BST)** is a binary tree with the ordering invariant: for every node N , **all keys in N 's left subtree are less than N 's key**, and **all keys in N 's right subtree are greater**. This lets you locate a key in $O(\log n)$ time on a balanced tree.

Insertion

Build a BST by inserting: **50, 30, 70, 20, 40, 60, 80**.

Insert 50: root is empty → 50 becomes the root.
 Insert 30: $30 < 50$ → go left (empty) → attach as left child of 50.
 Insert 70: $70 > 50$ → go right (empty) → attach as right child of 50.
 Insert 20: $20 < 50$ → left to 30; $20 < 30$ → left (empty) → attach as left child of 30.
 Insert 40: $40 < 50$ → left to 30; $40 > 30$ → right (empty) → attach as right child of 30.
 Insert 60: $60 > 50$ → right to 70; $60 < 70$ → left (empty) → attach as left child of 70.
 Insert 80: $80 > 50$ → right to 70; $80 > 70$ → right (empty) → attach as right child of 70.

Binary Search Tree — Insertion Order 50, 30, 70, 20, 40, 60, 80



Searching a BST

Search for 40:
 50? $40 < 50$ → go left
 30? $40 > 30$ → go right
 40? found ✓ (3 comparisons vs 7 linear)

For n items in a balanced BST, search takes $O(\log n)$. In a pathological (fully unbalanced) BST it degrades to $O(n)$ — the same as a linked list.

Exam Tip: If OCR asks you to "draw the BST formed by inserting the sequence X, Y, Z, ...", work strictly *left-to-right* through the sequence, applying the $</>$ rule from the root each time. Never re-balance.

Graphs

A **graph** is a set of **vertices (nodes)** connected by **edges**. Edges may be **directed** (one-way arrows) or **undirected**, **weighted** (labelled with a cost) or **unweighted**. Graphs model networks of all kinds: roads, web-page links, social relationships, dependencies.

Graph Terminology

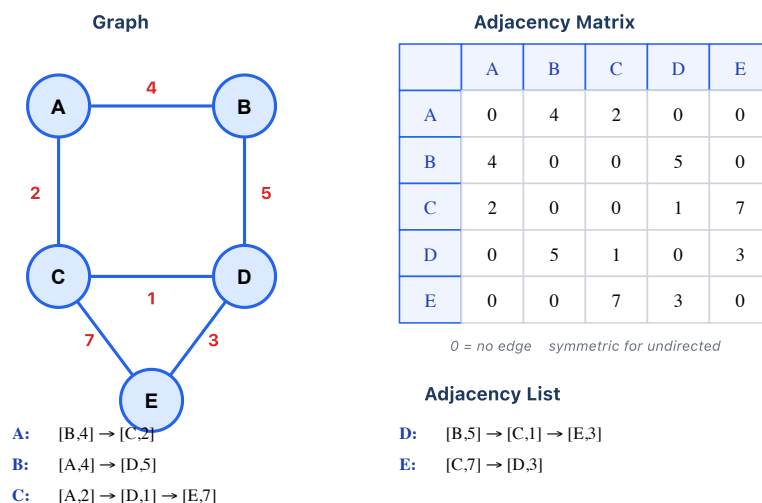
Term	Meaning
Vertex / node	A data point in the graph
Edge / arc	A connection between two vertices
Directed graph (digraph)	Edges have direction ($A \rightarrow B \neq B \rightarrow A$)
Undirected graph	Edges have no direction ($A - B = B - A$)
Weighted	Each edge has an associated number (distance, cost)
Cyclic	Contains at least one cycle (path returning to start)
Acyclic	No cycles. A DAG = directed acyclic graph.
Connected	A path exists between every pair of vertices
Disconnected	At least one vertex unreachable from another
Degree	Number of edges incident to a vertex (in-degree/out-degree for digraph)

Representations

Two standard representations; choose based on density.

- **Adjacency matrix** — a $V \times V$ table where cell (i,j) holds the weight of the edge from i to j (or $0/\infty$ for no edge). Space $O(V^2)$. Fast edge existence check $O(1)$. Wasteful for sparse graphs.
- **Adjacency list** — for each vertex, store a list of its neighbours (with weights). Space $O(V + E)$. Faster iteration over edges; slower "is there an edge?" check $O(\text{degree})$.

Weighted Graph with Adjacency Matrix and Adjacency List



Space $O(V+E)$ — efficient for sparse graphs

Property	Adjacency matrix	Adjacency list
Space	$O(V^2)$	$O(V + E)$
Edge existence	$O(1)$	$O(\text{degree})$
Iterate neighbours	$O(V)$	$O(\text{degree})$
Best for	Dense graphs	Sparse graphs

Hash Tables

A **hash table** stores key-value pairs in an array of "buckets". A **hash function** maps a key to an integer index in the array in (on average) $O(1)$ time, giving $O(1)$ average insertion, deletion and lookup.

Hash Function

A hash function $h(k)$ converts a key k into an index $0 \leq h(k) < m$ for an array of size m . Desirable properties:

- **Deterministic** — same key always produces the same index.
- **Uniform distribution** — keys spread evenly to minimise clusters.
- **Fast to compute** — ideally $O(1)$.
- **Avalanche effect** (for cryptographic hashes) — small input change → large output change.

Simple example: $h(k) = k \bmod m$. With $m = 7$ and keys 17, 23, 38, 45, 10:

```

h(17) = 17 mod 7 = 3
h(23) = 23 mod 7 = 2
h(38) = 38 mod 7 = 3    ← collision with 17
h(45) = 45 mod 7 = 3    ← collision with 17, 38
h(10) = 10 mod 7 = 3    ← collision again
  
```

Collisions

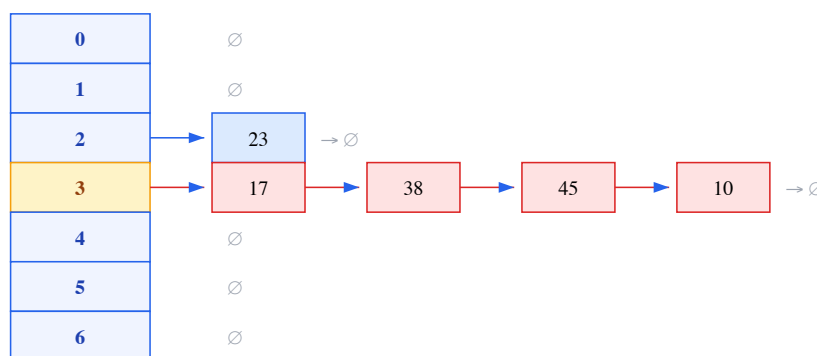
A **collision** occurs when two different keys hash to the same bucket. Every useful hash table must have a collision resolution strategy.

Collision Handling Strategies

Strategy	Method	Comment
Separate chaining	Each bucket holds a linked list (or another small container). On collision, append to the list.	Simple, handles arbitrary load factor, but uses extra pointer memory and can degrade to $O(n)$ if one chain grows huge.
Linear probing	On collision, try $(h(k)+1) \bmod m$, $(h(k)+2) \bmod m$, ... until a free slot is found.	Cache-friendly but suffers primary clustering — long runs of occupied slots.
Quadratic probing	Try $(h(k)+1^2) \bmod m$, $(h(k)+2^2) \bmod m$, $(h(k)+3^2) \bmod m$, ...	Reduces primary clustering at the cost of possible unreachable slots; table size should be prime.
Double hashing	Use a second hash: $(h_1(k) + i \cdot h_2(k)) \bmod m$.	Best spread, no clustering. Slightly slower to compute.

Worked Example: Chaining with keys 17, 23, 38, 45, 10

Hash Table with Separate Chaining ($m = 7$, $h(k) = k \bmod 7$)



Keys 17, 38, 45 and 10 all hash to index 3 and chain in insertion order.

Long chains degrade lookup toward $O(n)$. Resize / rehash when load factor gets high.

Same Example with Linear Probing

$m = 7$. Insert 17, 23, 38, 45, 10.

Insert 17 $\rightarrow h=3$. Slot 3 empty $\rightarrow$ place. $[-,-,-,17,-,-,-]$

Insert 23 $\rightarrow h=2$. Slot 2 empty $\rightarrow$ place. $[-,-,23,17,-,-,-]$

Insert 38 $\rightarrow h=3$. Slot 3 full; try 4. Empty $\rightarrow$ place. $[-,-,23,17,38,-,-]$

Insert 45 $\rightarrow h=3$. 3 full, 4 full; try 5. Empty $\rightarrow$ place. $[-,-,23,17,38,45,-]$

Insert 10 $\rightarrow h=3$. 3,4,5 full; try 6. Empty $\rightarrow$ place. $[-,-,23,17,38,45,10]$

Load Factor and Rehashing

The **load factor** $\lambda = n / m$ is the ratio of stored items to bucket count. Most implementations grow and **rehash** (typically doubling m , reinserting all keys) once λ exceeds 0.7–0.75 for open addressing, or some higher value for chaining. Rehashing is $O(n)$ but amortises to $O(1)$ per operation.

Hash tables are the implementation behind Python's `dict`, Java's `HashMap`, C++'s `unordered_map` and JavaScript's `Object` / `Map`.

Exam Tip: The standard OCR stem: "Explain why a hash table offers faster lookup than a linear search of an array." Answer: lookup is $O(1)$ average time because the hash function computes the index directly without comparing against other keys, whereas linear search is $O(n)$.

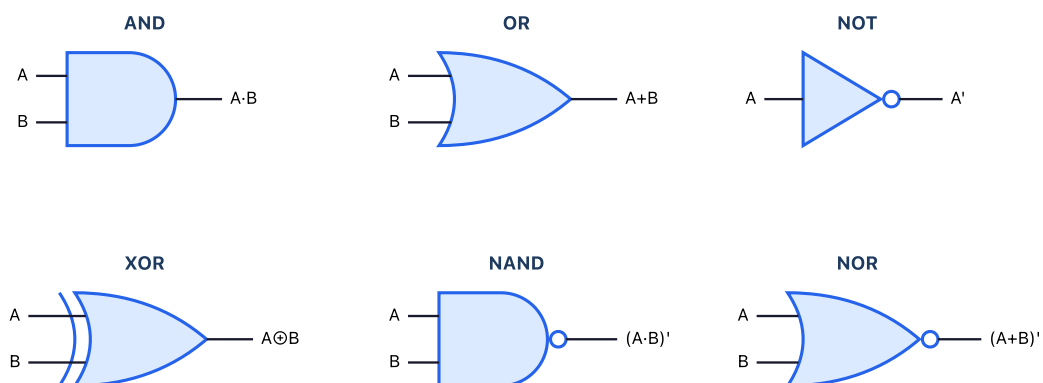
1.4.3 Boolean Algebra

Boolean algebra is the mathematics of two-valued logic — 0 and 1, false and true. It is the theoretical foundation on which **every digital circuit** is built.

Logic Gates

A **logic gate** is a physical device (usually a small arrangement of transistors) that implements one Boolean function. The six gates required by H446 are AND, OR, NOT, XOR, NAND and NOR. Each has a standard graphical symbol and a truth table.

Standard Logic Gate Symbols



Key features

- The small circle ("bubble") on NOT, NAND and NOR indicates negation of the output.
- AND is drawn with a flat back and rounded D-shape; OR has a curved back.
- XOR is OR with an extra curved input line.
- NAND and NOR are each

functionally complete — any Boolean expression can be built from just one of them. • NAND is the gate of choice in CMOS because it uses the fewest transistors.

Truth Tables

A	B	A AND B ($A \cdot B$)	A OR B ($A + B$)	A XOR B ($A \oplus B$)	A NAND B	A NOR B
0	0	0	0	0	1	1
0	1	0	1	1	1	0
1	0	0	1	1	1	0
1	1	1	1	0	0	0

A	NOT A (A')
0	1
1	0

Boolean Notation

Meaning	Algebraic	Alternative
AND	$A \cdot B$	$AB, A \wedge B$
OR	$A + B$	$A \vee B$
NOT	A'	$\neg A, \neg A, !A$
XOR	$A \oplus B$	$A \text{ XOR } B$
Constants	0, 1	false, true

Operator precedence (highest first): **NOT** → **AND** → **OR**. Parentheses override.

Example: build an expression from the English statement "F is true when it is either *sunny and warm* or *not raining*". Let S = sunny, W = warm, R = raining. Then $F = S \cdot W + R'$.

Boolean Identities

These laws let you simplify expressions algebraically. You should know each by name.

Law	AND form	OR form
Identity	$A \cdot 1 = A$	$A + 0 = A$

Null (Annihilator)	$A \cdot 0 = 0$	$A + 1 = 1$
Idempotent	$A \cdot A = A$	$A + A = A$
Complement	$A \cdot A' = 0$	$A + A' = 1$
Double negation	$(A')' = A$	
Commutative	$A \cdot B = B \cdot A$	$A + B = B + A$
Associative	$(A \cdot B) \cdot C = A \cdot (B \cdot C)$	$(A + B) + C = A + (B + C)$
Distributive	$A \cdot (B + C) = A \cdot B + A \cdot C$	$A + B \cdot C = (A + B) \cdot (A + C)$
Absorption	$A \cdot (A + B) = A$	$A + A \cdot B = A$
De Morgan's	$(A \cdot B)' = A' + B'$	$(A + B)' = A' \cdot B'$

De Morgan's Laws

De Morgan's laws: $(A \cdot B)' = A' + B'$ and $(A + B)' = A' \cdot B'$. Informally: to negate a product, negate each factor and change AND to OR. To negate a sum, negate each term and change OR to AND.

Proof by Truth Table: $(A + B)' = A' \cdot B'$

A	B	A+B	$(A+B)'$	A'	B'	$A' \cdot B'$
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

The $(A+B)'$ column equals the $A' \cdot B'$ column for every input — the law is proven.

Simplification Worked Examples

Example 1 (easy): Simplify $A \cdot B + A \cdot B'$

$$\begin{aligned}
 &A \cdot B + A \cdot B' \\
 &= A \cdot (B + B') && \text{[distributive]} \\
 &= A \cdot 1 && \text{[complement]} \\
 &= A && \text{[identity]}
 \end{aligned}$$

Example 2: Simplify $A + A \cdot B$

$$\begin{aligned}
 &A + A \cdot B \\
 &= A \cdot 1 + A \cdot B && \text{[identity]}
 \end{aligned}$$

```

= A · (1 + B)      [distributive]
= A · 1            [null: 1 + anything = 1]
= A                [identity]
(Or apply absorption directly.)

```

Example 3: Simplify $(A+B) \cdot (A+B')$

```

(A+B) · (A+B')
= A + B · B'      [OR-distributive: A + XY when common A]
= A + 0           [complement]
= A               [identity]

```

Example 4 (harder): Simplify $A \cdot B \cdot C + A \cdot B \cdot C' + A \cdot B' \cdot C$

```

= A · B · (C + C') + A · B' · C      [factor first two]
= A · B · 1 + A · B' · C              [complement]
= A · B + A · B' · C                  [identity]
= A · (B + B' · C)                    [factor A]
= A · (B + C)                         [absorption variant: X + X'Y = X + Y]

```

Exam Tip: For multi-mark simplification questions, **show each step with the law name**. OCR mark schemes credit a working line even if the final answer is wrong. The identity $X + X'Y = X + Y$ is very commonly useful.

Karnaugh Maps (K-maps)

A **Karnaugh map** is a graphical technique for simplifying Boolean functions of up to 4 (H446) or 5–6 (beyond spec) variables. The 2^n cells are arranged so that **adjacent cells differ in exactly one variable** (Gray-code ordering). Groupings of 1s correspond to simpler product terms; larger groups eliminate more variables.

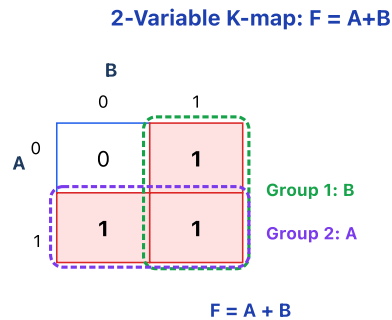
Rules for K-map Grouping

1. Groups must contain **powers of 2** cells: 1, 2, 4, 8, 16 — never 3, 5, 6, 7.
2. Groups should be **as large as possible**. Larger groups → fewer variables in the product term.
3. Every 1 must be in **at least one group**.
4. Groups may overlap.
5. Groups may **wrap around edges** (left↔right, top↔bottom) because the map is topologically a torus.
6. Read each group by noting which variables are constant and writing them; variables that change are eliminated.

Exam Tip: OCR loves to trap students into drawing a group of 3. **Never**. Redraw as a group of 2 plus another group of 2.

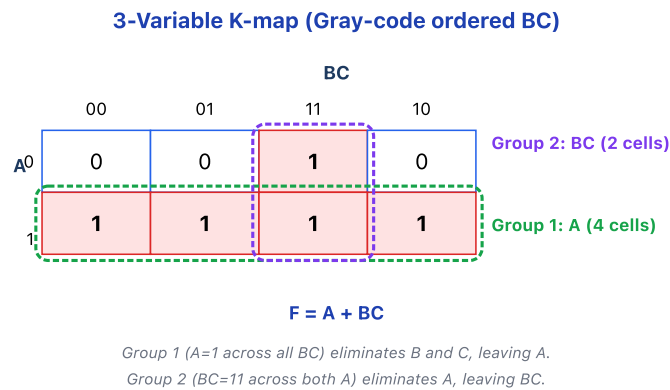
2-Variable K-map

Simplify $F(A,B)$ which is 1 when $(A,B) = (0,1), (1,0), (1,1)$.



3-Variable K-map

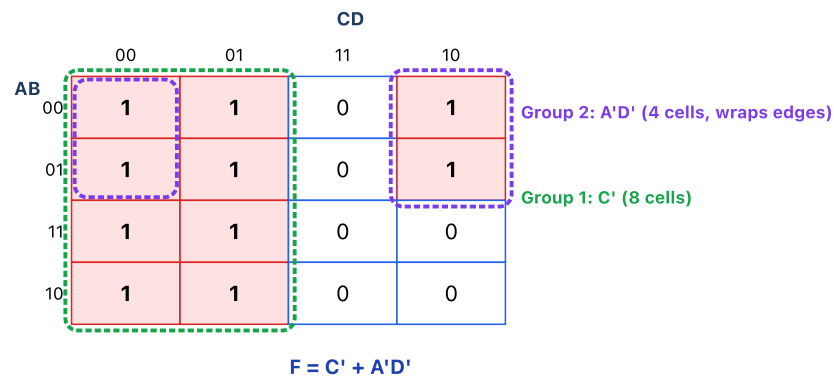
Simplify $F(A,B,C)$ given by the truth table where $F = 1$ for the minterms $A'BC$, $AB'C$, $AB'C$, ABC , ABC .



4-Variable K-map — Full Worked Example

Simplify $F(A,B,C,D) = \Sigma(0,1,2,4,5,6,8,9,12,13)$. (The Σ notation lists the minterm indices where $F = 1$.)

Minterm index = binary value of ABCD. So 0 = 0000, 1 = 0001, 2 = 0010, 4 = 0100, 5 = 0101, 6 = 0110, 8 = 1000, 9 = 1001, 12 = 1100, 13 = 1101.

4-Variable K-map — $F(A,B,C,D) = \Sigma(0,1,2,4,5,6,8,9,12,13)$ 

Group 1 covers the 8 cells where $C=0$ (A,B,D all vary) $\rightarrow$ the term is just C' .

Group 2 wraps $CD=00$ and $CD=10$ in rows $AB=00, 01$ (minterms 0, 2, 4, 6): $A=0, D=0 \rightarrow A'D'$.

Every 1 is covered; no group of size 3 is used. Groups may wrap the edges.

Gray-code ordering: 00, 01, 11, 10 — not 00, 01, 10, 11.

Only adjacent orderings work; one variable changes per step. Otherwise adjacent cells would not differ by exactly one variable.

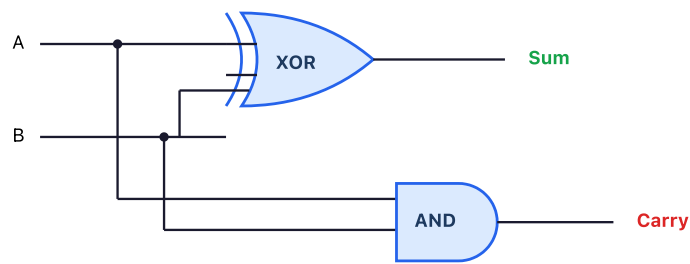
Exam Tip: When reading a K-map group: list the variables whose value is *constant* within the group. Variables that take both 0 and 1 in the group are eliminated. Bigger groups $\Rightarrow$ more eliminated $\Rightarrow$ simpler term.

Adders

Half Adder

A **half adder** adds two single bits A and B , producing a 2-bit result: a **Sum** (least significant bit) and a **Carry** (most significant bit). **Sum** = $A \oplus B$, **Carry** = $A \cdot B$. It cannot accept an incoming carry, so only the least-significant bit of a multi-bit adder can be a half adder.

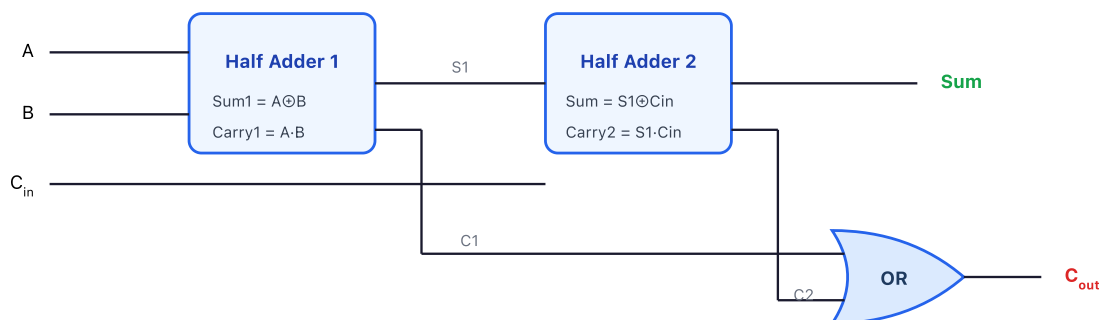
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Half Adder — $\text{Sum} = A \oplus B$, $\text{Carry} = A \cdot B$ 

Full Adder

A **full adder** adds **three** single-bit inputs: A, B and C_{in} (a carry from the previous stage). It outputs a **Sum** bit and a **C_{out}** . $\text{Sum} = A \oplus B \oplus C_{in}$; $C_{out} = A \cdot B + C_{in} \cdot (A \oplus B)$. A full adder can be built from two half adders plus an OR gate.

A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Full Adder = $2 \times \text{Half Adder} + \text{OR}$ 

Ripple-Carry Adder

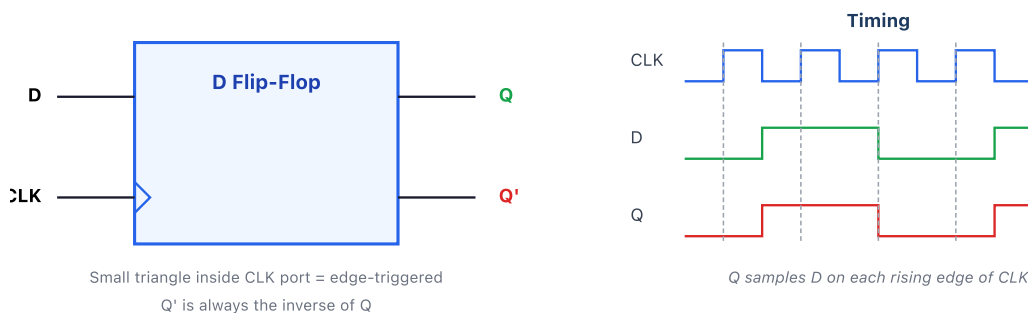
To add two n -bit numbers, connect n full adders in a chain: C_{out} of stage i becomes C_{in} of stage $i+1$. The first stage (LSB) can be a half adder (no carry in). This is called a **ripple-carry adder** because the carry ripples from LSB to MSB through every full adder, making propagation delay $O(n)$. Faster designs (carry-lookahead) exist but are beyond spec.

D-Type Flip-Flop

A **flip-flop** is a bistable electronic circuit that stores one bit of state. A **D-type flip-flop** has one data input (**D**), one clock input (**CLK**) and two outputs (**Q** and its complement **Q'**). On the **rising edge** of the clock, the value on D is *captured* and appears at Q. Between clock edges Q holds its value, so the flip-flop acts as a **1-bit memory cell**.

D	CLK	Q (next)	Notes
0	↑	0	Rising edge, D=0 captured.
1	↑	1	Rising edge, D=1 captured.
X	0 / 1 / ↓	Q (held)	No edge → output unchanged.

D-Type Flip-Flop (Edge-Triggered)



Because an n -bit register is simply n D-flip-flops sharing one clock, the D-type is the fundamental building block of CPU registers, cache lines, DRAM refresh latches, shift registers and state machines.

SR Flip-Flop (for Comparison)

The **SR (Set-Reset) flip-flop** has two inputs S and R. Setting $S = 1$ forces $Q = 1$ (set); setting $R = 1$ forces $Q = 0$ (reset); $S = R = 0$ holds state; **$S = R = 1$ is forbidden** because the two outputs would both be forced, leaving the device in an indeterminate state when the inputs are released. This "race" problem motivates the D-type: it has only one data input and therefore cannot enter the forbidden state.

S	R	Q (next)	Comment
0	0	Q (held)	Store

0	1	0	Reset
1	0	1	Set
1	1	—	Forbidden

Exam Tip: Common OCR question: "Explain why a D-type flip-flop is used in preference to an SR flip-flop." Answer: a D-type has a single data input, so the invalid $S=R=1$ state cannot occur, and its edge-triggered behaviour synchronises reliably with a system clock.

Summary — What to Memorise for H446

- Five primitive types and their typical sizes.
- Convert denary $\leftrightarrow$ binary $\leftrightarrow$ hex fluently using nibble grouping.
- Two's complement: flip+add 1; MSB has weight -2^{n-1} ; range $-2^{n-1}..+2^{n-1}-1$; subtraction via addition of the complement.
- Overflow = signed result outside range; carry = unsigned carry out of MSB.
- Floating-point: sign/exponent/mantissa; normalised mantissas begin 0.1... (positive) or 1.0... (negative); range set by exponent, precision by mantissa.
- Bitwise AND clears, OR sets, XOR toggles; arithmetic right shift preserves the sign bit.
- ASCII is 7-bit / 128 chars; Unicode is universal with UTF-8/16/32 encodings.
- Arrays static & zero-indexed; linked lists dynamic with pointers; stacks LIFO (push/pop); queues FIFO (enqueue/dequeue); circular queue wraps with mod.
- BSTs keep left<parent<right; in-order traversal returns sorted values.
- Graphs: adjacency matrix $O(V^2)$ space, $O(1)$ edge check; adjacency list $O(V+E)$ space, good for sparse graphs.
- Hash tables: $O(1)$ average lookup; collisions handled by chaining or open addressing (linear, quadratic, double).
- Know all six gate symbols, their truth tables, and that NAND/NOR are functionally complete.
- Memorise Boolean laws by name: identity, null, idempotent, complement, absorption, De Morgan's.
- K-map rules: Gray-code ordering; groups must be powers of 2; largest possible; may wrap; every 1 in at least one group; no groups of 3.
- Half adder: $\text{Sum} = A \oplus B$, $\text{Carry} = A \cdot B$. Full adder: three inputs, built from two HAs + OR.
- D-type flip-flop: 1-bit memory, edge-triggered; basis of all registers.

Exam Tip: Time-pressure tip: when a Paper 1 question is worth 6–8 marks on this topic (typical for K-map or BST questions), allocate ~1 minute per mark. Always **show working**: conversion steps, law names, group boundaries drawn clearly on the K-map, probe sequence for a hash table. Correct working with a wrong answer often scores more than a bare right answer.

